Рубежный контроль 1 по курсу ПиКЯП
Вариант 31-Г
Выполнила: Саргсян Виктория, ИУ5-35Б

Предметная область (классы 1 и 2)

31	Таблица данных	База данных

Вариант Г.

1. «Отдел» и «Сотрудник» связаны соотношением один-ко-многим. Выведите список всех отделов, у которых название начинается с буквы «А», и список работающих в них сотрудников.
2. «Отдел» и «Сотрудник» связаны соотношением один-ко-многим. Выведите список отделов с максимальной зарплатой сотрудников в каждом отделе, отсортированный по максимальной зарплате.
3. «Отдел» и «Сотрудник» связаны соотношением многие-ко-многим. Выведите список всех связанных сотрудников и отделов, отсортированный по отделам, сортировка по сотрудникам произвольная.

Текст программы:

```
# -*- coding: utf-8 -*-
"""
РК1 по ПиКЯП. Вариант 31-Г.

Предметная область:
Класс 1 – «Таблица данных»
Класс 2 – «База данных»

Связи:
    «База данных» – «Таблица данных»: один-ко-многим (поле db_id в таблице);
    «База данных» – «Таблица данных»: многие-ко-многим (класс-связка
DatabaseTable).
"""

from operator import itemgetter

"Класс «Таблица данных»"
class DataTable:

    def __init__(self, id, name, rows_count, db_id):
        self.id = id                    #ID записи о таблице
        self.name = name                #Название таблицы
        self.rows_count = rows_count    #Количественный признак: число записей в
таблице
        self.db_id = db_id              #ID базы данных
```

```
"Класс «База данных»"
class Database:

    def __init__(self, id, name):
        self.id = id      #ID записи о базе данных
        self.name = name  #Название базы данных

"Класс «Таблицы базы данных» (связь многие-ко-многим)"
class DatabaseTable:

    def __init__(self, db_id, table_id):
        self.db_id = db_id        # ID базы данных
        self.table_id = table_id  # ID таблицы данных

databases = [
    Database(1, 'Архив документов'),
    Database(2, 'Бухгалтерия'),
    Database(3, 'Авиарейсы'),
    Database(4, 'Склад'),
    Database(5, 'Аналитика продаж'),
]

tables = [
    DataTable(1, 'Договоры', 1200, 1),
    DataTable(2, 'Сканы', 5400, 1),
    DataTable(3, 'Проводки', 8700, 2),
    DataTable(4, 'Счета', 950, 2),
    DataTable(5, 'Рейсы', 3100, 3),
    DataTable(6, 'Пассажиры', 15000, 3),
    DataTable(7, 'Товары', 640, 4),
    DataTable(8, 'Поставщики', 85, 4),
    DataTable(9, 'Продажи', 22000, 5),
]

# Связь многие-ко-многим: одна таблица может использоваться в нескольких базах
database_tables = [
    DatabaseTable(1, 1), DatabaseTable(1, 2), DatabaseTable(1, 4),
    DatabaseTable(2, 3), DatabaseTable(2, 4),
    DatabaseTable(3, 5), DatabaseTable(3, 6),
    DatabaseTable(4, 7), DatabaseTable(4, 8), DatabaseTable(4, 9),
    DatabaseTable(5, 3), DatabaseTable(5, 6), DatabaseTable(5, 7),
DatabaseTable(5, 9),
]


def query_1():
    """
    1 (один-ко-многим): все базы данных, название которых начинается
    с буквы А, и список таблиц в них.
    """
    return {
```

```
        db.name: [t.name for t in tables if t.db_id == db.id]
        for db in databases
        if db.name.startswith('A')
    }


def query_2():
    """
    2 (один-ко-многим): список баз данных с max количеством
    записей в таблицах каждой базы, отсортированный по этому max (по убыванию).
    """
    # max только для баз, у которых есть хотя бы одна таблица
    max_rows = {
        db.id: max(t.rows_count for t in tables if t.db_id == db.id)
        for db in databases
        if any(t.db_id == db.id for t in tables)
    }
    result = [(db.name, max_rows[db.id]) for db in databases if db.id in
max_rows]
    # сорт. по макс. количеству записей
    return sorted(result, key=itemgetter(1), reverse=True)
    #можно сделать по возрастанию, убрав reverse=True!

def query_3():
    """
    3 (многие-ко-многим): список всех связанных таблиц и баз данных,
    отсортированный по базам данных (порядок таблиц произвольный).
    """
    pairs = [
        (db.name, t.name)
        for link in database_tables
        for db in databases if db.id == link.db_id
        for t in tables if t.id == link.table_id
    ]
    # сорт. по названию базы данных
    return sorted(pairs, key=itemgetter(0))


def main():
    print('Задание 1. Базы данных на букву «А» и их таблицы')
    for db_name, table_names in query_1().items():
        print(f'  {db_name}: {", ".join(table_names)}')

    print('\nЗадание 2. Базы данных по максимальному числу записей в таблице')
    for db_name, max_rows in query_2():
        print(f'  {db_name}: {max_rows}')

    print('\nЗадание 3. Связанные базы данных и таблицы (многие-ко-многим)')
    for db_name, table_name in query_3():
        print(f'  {db_name} — {table_name}')
```

```python
if __name__ == '__main__':
    main()
```

Результат выполнения программы:

```
mathv@DESKTOP-RJ5M4RC:~/SargsyanVS-Paradigms-Labs-2026/RK1$ python3 main.py
Задание 1. Базы данных на букву «А» и их таблицы
  Архив документов: Договоры, Сканы
  Авиарейсы: Рейсы, Пассажиры
  Аналитика продаж: Продажи

Задание 2. Базы данных по максимальному числу записей в таблице
  Аналитика продаж: 22000
  Авиарейсы: 15000
  Бухгалтерия: 8700
  Архив документов: 5400
  Склад: 640

Задание 3. Связанные базы данных и таблицы (многие-ко-многим)
  Авиарейсы — Рейсы
  Авиарейсы — Пассажиры
  Аналитика продаж — Проводки
  Аналитика продаж — Пассажиры
  Аналитика продаж — Товары
  Аналитика продаж — Продажи
  Архив документов — Договоры
  Архив документов — Сканы
  Архив документов — Счета
  Бухгалтерия — Проводки
  Бухгалтерия — Счета
  Склад — Товары
  Склад — Поставщики
  Склад — Продажи
```